

Functional Requirements Document (FRD): Algorithmic Trading System

1. Introduction

1.1 Purpose of the Document

This Functional Requirements Document (FRD) provides a detailed description of the functional requirements for the Algorithmic Trading System. It is intended for developers, testers, and technical stakeholders to understand what needs to be built and to ensure the system meets the business objectives outlined in the Business Requirements Document (BRD). The FRD serves as a blueprint for the system's design, development, and testing phases, ensuring that all functional aspects are clearly defined and aligned with user needs.

1.2 Scope

The scope of this FRD includes all functional aspects of the Algorithmic Trading System, such as:

- Trading capabilities across multiple asset classes
- Real-time data processing and analytics
- AI-driven natural language interaction and code generation
- User interfaces for both non-technical and professional users
- Security, compliance, and auditability features
- Integration with external systems and services

This document does not cover non-functional requirements such as performance, scalability, or reliability in exhaustive detail, though they are addressed briefly where relevant to functionality.

2. System Overview

The Algorithmic Trading System is an enterprise-grade platform designed to empower users to develop, test, and execute trading strategies across multiple

asset classes, including stocks, ETFs, futures, options, forex, and cryptocurrencies. Built on a "Best-of-Breed" integration strategy, the system leverages leading open-source technologies to deliver a modular, scalable, and high-performance solution.

Key features include:

- **AI-Powered Assistant:** Provides natural language interaction for trading insights, strategy generation, and code debugging.
- **Real-Time Data Processing:** Ingests and processes market data with low latency to support high-frequency trading.
- **Advanced Analytics:** Offers comprehensive backtesting, forecasting, and portfolio optimization tools.
- **Intuitive User Interfaces:** Includes a real-time dashboard and a no-code strategy builder for diverse user personas.
- **Enterprise-Grade Security:** Implements Zero-Trust architecture, Role-Based Access Control (RBAC), encryption, and audit trails to ensure compliance and data protection.

The system is architected using technologies such as NautilusTrader, Apache Kafka, LangChain, Next.js, and FastAPI, ensuring flexibility, performance, and ease of integration.

3. Functional Requirements

The functional requirements are organized by system module to provide clarity and structure. Each requirement is uniquely identified and described in detail to ensure testability and clarity.

3.1 Core Trading Engine

The Core Trading Engine is the heart of the system, responsible for executing trading strategies, managing orders, and interacting with brokers.

- **FR-CE-001:** The system shall support trading of the following asset classes: stocks, ETFs, futures, options, forex, and cryptocurrencies.
- **FR-CE-002:** The system shall allow users to define custom trading strategies using Python code, supporting technical indicators (e.g., SMA, RSI), signals,

and order management logic.

- **FR-CE-003:** The system shall provide a backtesting engine that enables users to test strategies against historical data, generating performance metrics such as profit/loss, Sharpe ratio, maximum drawdown, and win/loss ratio.
- **FR-CE-004:** The system shall support live trading, connecting to brokers (e.g., Interactive Brokers) for order execution and position management using real-time data feeds.
- **FR-CE-005:** The system shall handle multiple order types, including market, limit, stop, and advanced orders such as Volume-Weighted Average Price (VWAP) and Time-Weighted Average Price (TWAP).
- **FR-CE-006:** The system shall maintain a real-time portfolio view, displaying current positions, unrealized profit/loss, margin usage, and account balances.
- **FR-CE-007:** The system shall provide order management features, allowing users to place, modify, and cancel orders programmatically or via the UI.
- **FR-CE-008:** The system shall support paper trading to allow users to test strategies in a simulated environment with real-time data before live deployment.

3.2 Data Pipelines

The Data Pipelines module manages the ingestion, processing, and storage of market and system data to ensure availability and consistency.

- **FR-DP-001:** The system shall ingest real-time market data from multiple sources, including Yahoo Finance, Alpha Vantage, and Interactive Brokers.
- **FR-DP-002:** The system shall process and normalize incoming data to ensure consistency across different sources and formats (e.g., aligning timestamps, handling missing values).
- **FR-DP-003:** The system shall store historical market data in a time-series database (e.g., ClickHouse) for efficient querying and analysis.
- **FR-DP-004:** The system shall implement a fallback mechanism to switch data sources automatically in case of provider outages or data quality issues, ensuring uninterrupted operation.
- **FR-DP-005:** The system shall use Apache Kafka for streaming market data, ensuring scalability, fault tolerance, and low-latency delivery to downstream components.

- **FR-DP-006:** The system shall log all system events, errors, and performance metrics for monitoring, debugging, and audit purposes.

3.3 Agentic AI Assistant

The Agentic AI Assistant enhances user productivity by providing natural language interaction, code generation, and debugging support.

- **FR-AI-001:** The AI assistant shall interpret natural language queries from users, providing responses related to market data (e.g., price trends), strategy suggestions, or system functions.
- **FR-AI-002:** The AI assistant shall generate executable Python code for trading strategies based on user descriptions (e.g., "Create a strategy that buys when RSI is below 30"), leveraging tools like OpenHands.
- **FR-AI-003:** The AI assistant shall debug and optimize existing strategy code, offering suggestions for performance improvements or error corrections (e.g., identifying infinite loops).
- **FR-AI-004:** The AI assistant shall integrate with a Retrieval-Augmented Generation (RAG) pipeline to retrieve and summarize information from user-uploaded documents (e.g., PDFs, CSVs).
- **FR-AI-005:** The AI assistant shall support multi-turn conversations, maintaining context across interactions to enable follow-up questions.

3.4 Prediction and Analytics

The Prediction and Analytics module provides advanced tools for forecasting, backtesting, and portfolio management.

- **FR-PA-001:** The system shall include machine learning models (e.g., LSTM, ARIMA) for forecasting asset prices and market trends based on historical and real-time data.
- **FR-PA-002:** The system shall provide backtesting tools with detailed metrics, including equity curves, trade logs, and risk analysis (e.g., Value at Risk, beta).
- **FR-PA-003:** The system shall offer portfolio optimization features using tools like PyPortfolioOpt to suggest optimal asset allocations based on user-defined risk and return preferences.
- **FR-PA-004:** The system shall detect anomalies in trading data and system performance using PyOD, alerting users to potential issues (e.g., sudden

price spikes, system errors).

- **FR-PA-005:** The system shall generate exportable reports (e.g., PDF, CSV) from backtests and analytics for user review and sharing.

3.5 User Interface

The User Interface module encompasses the frontend components, designed to be intuitive and accessible for diverse users.

- **FR-UI-001:** The system shall provide a real-time dashboard displaying portfolio status, live charts, and risk metrics.
- **FR-UI-002:** The dashboard shall use react-financial-charts for interactive charting, supporting multiple timeframes (e.g., 1m, 5m, 1h, 1d) and technical indicators (e.g., SMA, RSI).
- **FR-UI-003:** The system shall include a no-code strategy builder with drag-and-drop functionality, using Blockly to create visual strategy workflows without requiring programming knowledge.
- **FR-UI-004:** The strategy builder shall generate executable Python code from visual configurations for deployment in the Core Trading Engine.
- **FR-UI-005:** The system shall have an AI chat interface for users to interact with the assistant via natural language, integrated into the dashboard.
- **FR-UI-006:** The UI shall support user authentication, session management, and role-based access controls, ensuring secure and personalized access.

3.6 API and Orchestration

The API and Orchestration module manages communication between system components and external services.

- **FR-AO-001:** The system shall use FastAPI as the central API bridge, exposing RESTful endpoints for all major functionalities (e.g., strategy deployment, data retrieval).
- **FR-AO-002:** The system shall implement gRPC for low-latency data streaming between internal services (e.g., data pipelines to trading engine).
- **FR-AO-003:** The API shall support OAuth2 with JSON Web Tokens (JWT) for secure authentication and authorization.
- **FR-AO-004:** The system shall provide API documentation automatically generated from code (e.g., using Swagger), accessible to developers.

3.7 Security and Compliance

The Security and Compliance module ensures the system meets security and regulatory standards.

- **FR-SC-001:** The system shall enforce Zero-Trust architecture, requiring verification for every access request, regardless of user location or device.
- **FR-SC-002:** The system shall implement Role-Based Access Control (RBAC) to manage user permissions based on roles (e.g., Trader, Analyst, Admin).
- **FR-SC-003:** All sensitive data (e.g., user credentials, trading strategies) shall be encrypted using AES-256, both at rest and in transit.
- **FR-SC-004:** The system shall maintain immutable audit trails of all trading activities and system changes using Apache Iceberg, ensuring regulatory compliance.
- **FR-SC-005:** The system shall comply with financial regulations (e.g., SEC, FINRA) and data privacy laws (e.g., GDPR, CCPA).

4. Use Cases

The following use cases illustrate how different users will interact with the system, providing concrete examples of functionality.

4.1 Use Case 1: Non-technical User Creating a Strategy

- **Actor:** Non-technical Trader (Sarah)
- **Description:** Sarah uses the no-code strategy builder to create a moving average crossover strategy without writing code.
- **Preconditions:** Sarah is authenticated and has access to the strategy builder.
- **Steps:**
 1. Sarah navigates to the strategy builder from the dashboard.
 2. She selects the "Moving Average Crossover" template from a list of pre-built options.
 3. She sets the short moving average period to 10 days and the long period to 50 days using sliders or input fields.
 4. She selects "Stocks" as the asset class and specifies "AAPL" as the instrument from a dropdown menu.

5. She clicks "Run Backtest" to test the strategy with historical data from the past year.
 6. The system displays backtest results, including profit/loss, Sharpe ratio, and an equity curve.
 7. Satisfied with the results, Sarah clicks "Deploy" to activate the strategy for live trading.
- **Postconditions:** The strategy is deployed and begins executing trades based on real-time data when conditions are met.

4.2 Use Case 2: Professional Trader Executing a Trade

- **Actor:** Professional Trader (Michael)
- **Description:** Michael executes a high-frequency trade using real-time data and programmatic access.
- **Preconditions:** Michael is authenticated and has a configured trading strategy.
- **Steps:**
 1. Michael views the real-time dashboard, monitoring price movements for a specific futures contract.
 2. He identifies an opportunity based on a custom signal and decides to place a market order.
 3. He uses the API to submit the order programmatically via a Python script.
 4. The system processes the order and executes it via the broker within 100ms.
 5. Michael receives confirmation of the trade execution on the dashboard.
 6. He reviews post-trade analytics, including profit/loss and execution slippage, to assess performance.
- **Postconditions:** The trade is executed, and portfolio and analytics data are updated in real-time.

4.3 Use Case 3: User Querying the AI Assistant

- **Actor:** User (any authenticated user)
- **Description:** The user asks the AI assistant for market insights to inform a trading decision.

- **Preconditions:** The user is authenticated and has access to the AI chat interface.
- **Steps:**
 1. The user opens the AI chat interface from the dashboard.
 2. The user types, "What is the current trend for AAPL?"
 3. The AI assistant processes the query, retrieving real-time and historical data for AAPL.
 4. The AI responds with a summary, e.g., "AAPL is in an uptrend, with a 5% increase over the past week, supported by a rising 20-day SMA."
 5. The user asks a follow-up question, "Should I buy now?"
 6. The AI provides a suggestion based on predefined risk parameters, e.g., "Based on current RSI of 65, it may be overbought; consider waiting for a pullback."
- **Postconditions:** The user receives actionable insights and can proceed with their decision-making process.

5. User Interface Requirements

The User Interface (UI) is designed to be intuitive, responsive, and tailored to both non-technical and professional users.

5.1 Real-Time Dashboard

- The dashboard shall display:
 - **Portfolio Summary:** Total value, cash balance, positions, and open orders, updated in real-time.
 - **Interactive Charts:** Candlestick, OHLC, and volume charts with zoom and pan capabilities, using react-financial-charts.
 - **Risk Metrics:** Value at Risk (VaR), Sharpe ratio, beta, and other key indicators, refreshed periodically.
 - **Order Blotter:** Real-time view of open, pending, and executed orders, filterable by status or asset.
- Charts shall support multiple timeframes (e.g., 1m, 5m, 1h, 1d) and technical indicators (e.g., SMA, RSI, MACD).

5.2 No-Code Strategy Builder

- The builder shall provide a drag-and-drop interface with pre-built blocks for:
 - **Data Inputs:** Price, volume, and other market data fields.
 - **Indicators:** SMA, RSI, Bollinger Bands, etc.
 - **Conditions:** Greater than, less than, crossover, etc.
 - **Actions:** Buy, sell, send alert, etc.
- Users shall be able to connect blocks to form a strategy flowchart, visually representing logic.
- The builder shall generate executable Python code from the visual configuration, viewable and editable by advanced users.
- Users shall have the ability to save, load, and share strategies via a library or export feature.

5.3 AI Chat Interface

- The chat interface shall allow users to interact with the AI assistant via text input in a conversational format.
- Responses shall be displayed in a scrollable chat history with timestamps.
- The interface shall support multi-turn conversations, maintaining context for at least 10 previous interactions.
- The UI shall include a "Clear Chat" option to reset the conversation.

6. Data Requirements

The system requires various types of data to function effectively, with specific needs for input, output, and storage.

6.1 Market Data

- **Description:** Real-time and historical price data for supported asset classes (stocks, ETFs, futures, options, forex, cryptocurrencies).
- **Details:** Shall include price, volume, and order book data (e.g., bid/ask spreads where available).
- **Sources:** Yahoo Finance, Alpha Vantage, Interactive Brokers, and other configured providers.

6.2 User Data

- **Description:** Data related to user accounts and activities.
- **Details:** User profiles (credentials, preferences), trading strategy configurations, historical performance data, portfolio and account information.
- **Storage:** Encrypted and segmented by user for security and privacy.

6.3 System Data

- **Description:** Operational data for monitoring and compliance.
- **Details:** Logs of system events, errors, performance metrics, and audit trails for trading activities.
- **Retention:** Configurable retention periods (e.g., 30 days for logs, 7 years for audit trails).

6.4 Data Storage Solutions

- **Relational Data:** Stored in PostgreSQL for structured user and configuration data.
- **Time-Series Data:** Stored in ClickHouse for efficient querying of market data.
- **Vector Embeddings:** Stored in Qdrant for AI-related tasks (e.g., RAG pipeline).
- **Immutable Logs:** Stored in Apache Iceberg for auditability and regulatory compliance.

7. Integration Requirements

The system must integrate seamlessly with internal components and external services to function as a cohesive platform.

- **NautilusTrader:** Shall subscribe to Kafka topics for market data and publish order events to the trading engine.
- **FastAPI:** Shall expose RESTful endpoints for frontend and AI services to interact with the backend (e.g., `/strategies` , `/portfolio`).

- **gRPC:** Shall be used for high-performance, low-latency communication between internal services (e.g., data pipelines to trading engine).
- **External Integrations:** Shall connect to brokers (e.g., Interactive Brokers) via FIX protocol or proprietary APIs for trade execution and account management.

8. Security Requirements

Security is critical for protecting user data and ensuring regulatory compliance.

- **Authentication:** Users shall authenticate using OAuth2 with JWT tokens, supporting multi-factor authentication (MFA) as an option.
- **Authorization:** RBAC shall control access to system features based on user roles (e.g., Trader: execute trades; Analyst: view analytics; Admin: manage users).
- **Encryption:** All sensitive data shall be encrypted using AES-256, and all communications shall use TLS 1.3.
- **Auditing:** The system shall log all access attempts, trades, and system changes in an immutable format using Apache Iceberg.
- **Vulnerability Management:** Regular scans shall be conducted using tools like Bandit to identify and remediate security issues.

9. Performance Requirements

The system must meet strict performance standards to support real-time trading and analytics.

- **Trade Execution Latency:** Less than 100ms from order placement to execution confirmation.
- **Data Ingestion Latency:** Less than 10ms per market data message processed.
- **AI Query Response Time:** Less than 2 seconds for natural language queries to the AI assistant.
- **Throughput:** Capable of processing up to 10,000 market data messages per second.

- **Concurrency:** Support for at least 1000 concurrent users without degradation in performance.

10. Scalability and Reliability

The system must be designed to handle growth and ensure continuous operation.

- **Scalability:** Services shall be containerized using Docker and scalable horizontally using Kubernetes, with load balancing to distribute traffic.
- **High Availability:** The system shall have no single points of failure and maintain 99.99% uptime, achieved through redundancy and failover mechanisms.
- **Disaster Recovery:** Regular backups shall be performed daily, with a recovery time objective (RTO) of 1 hour and a recovery point objective (RPO) of 5 minutes.

11. Compliance and Regulatory Requirements

The system must adhere to relevant financial and data privacy regulations.

- **Financial Regulations:** Shall comply with SEC, FINRA, and other applicable regulations for trading platforms (e.g., record-keeping, reporting).
- **Data Privacy:** Shall adhere to GDPR, CCPA, and other data protection laws for handling user data, including the right to be forgotten.
- **Audit Trails:** Shall provide tamper-proof logs of all trading activities and system changes, accessible for regulatory inspections.

12. Testing and Quality Assurance

A comprehensive testing strategy ensures the system meets all functional requirements.

- **Unit Testing:** Each module shall have unit tests covering at least 80% of the code, using frameworks like pytest.

- **Integration Testing:** Shall verify interactions between services (e.g., data pipelines to trading engine) and external systems (e.g., brokers).
- **Performance Testing:** Shall simulate peak loads (e.g., 10,000 messages/second) to ensure latency and throughput targets are met.
- **Security Testing:** Shall include penetration testing and regular vulnerability scans to identify weaknesses.
- **User Acceptance Testing (UAT):** Shall involve end-users to validate the system against business requirements and use cases.

13. Appendices

13.1 Glossary

- **Agentic AI:** An AI system capable of autonomous decision-making and action based on user input.
- **RAG Pipeline:** Retrieval-Augmented Generation, a method for enhancing AI responses with retrieved information from a knowledge base.
- **Zero-Trust Architecture:** A security model that assumes no trust by default and requires verification for every access request.

13.2 References

- NautilusTrader Documentation: <https://nautilustrader.io/docs/>
- Apache Kafka Documentation: <https://kafka.apache.org/documentation/>
- LangChain Documentation: <https://langchain.com/docs/>

This FRD provides a detailed and elaborate outline of the functional requirements for the Algorithmic Trading System, ensuring that all stakeholders have a clear understanding of what the system must achieve. It serves as a critical reference for the design, development, and testing phases of the project.